

Algoritmos y Estructuras de Datos.

- Clase 7 - Complejidad Computacional -

Ejercicios.

Ejercicio 1:

Para cada uno de los algoritmos presentados:

- Expresar en función de n el tiempo de ejecución.
- Analizar y resolver la correspondiente recurrencia.
- Determinar el orden de las funciones obtenidas usando notación big-Oh.
- Comparar el tiempo de ejecución de la función $rec1$ con el de la función $rec2$.
- Implementar un algoritmo más eficiente que la función f (es decir que el $T(n)$ sea menor).

```
In [ ]: 1 # i)
        2 def rec1(n):
        3     if (n <= 1):
        4         return 1
        5     else:
        6         return rec1 (n-1) + rec1 (n-1)
```

```
In [ ]: 1 # ii)
        2 def rec2(n):
        3     if (n <= 1):
        4         return 1
        5     else:
        6         return 2 * rec2(n-1)
```

```
In [ ]: 1 # iii)
        2 def f(n):
        3     if (n==0):
        4         return 0
        5     elif ( n == 1 ):
        6         return 1
        7     else:
        8         return f(n-2) * f (n-2)
        9
```

```
In [ ]: 1 # iv)
        2 def potencia_iter (x, n):
        3     if (n==0):
        4         potencia = 1
        5     elif (n==1):
        6         potencia = x
        7     else:
        8         potencia = x
        9         for i in range(2,n):
        10             potencia = potencia * x
        11     return potencia
```

```
In [ ]: 1 # v)
2 def potencia_rec(x, n):
3     if (n==0):
4         return 1
5     elif (n==1):
6         return x
7     elif (n mod 2) == 0:
8         return potencia_rec(x * x, n div 2)
9     else:
10        return potencia_rec(x * x, n div 2) * x
```

Ejercicio 2:

- Implementar un algoritmo más eficiente que la función `f` (es decir que el $T(n)$ sea menor).
- Comparar el tiempo de ejecución de la función `f` con el de la nueva función.

Ejercicio 3:

- Resolver las siguientes recurrencias:

a)

$$T(n) = \begin{cases} 1 & \text{si } n = 1, \\ 8 T(\frac{n}{2}) + n^3 & \text{si } n \geq 2. \end{cases}$$

b)

$$T(n) = \begin{cases} c & \text{si } n = 0, \\ d & \text{si } n = 1, \\ [T(n-2)]^2 & \text{si } n \geq 2. \end{cases}$$

c)

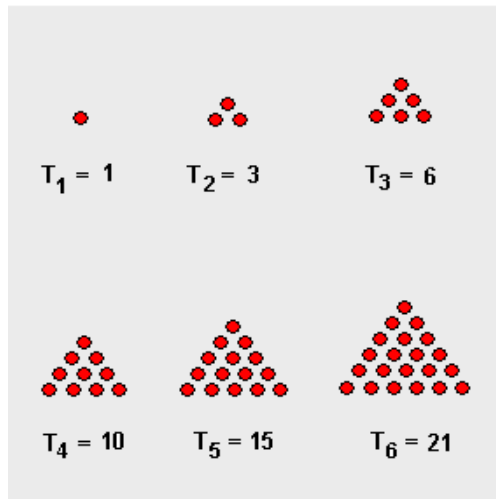
$$T(n) = \begin{cases} 2 & \text{si } n = 1, \\ T(n-1) + n & \text{si } n \geq 2. \end{cases}$$

d)

$$T(n) = \begin{cases} 2 & \text{si } n = 1, \\ T(n-1) + \frac{n}{2} & \text{si } n \geq 2. \end{cases}$$

Ejercicio 4:

Los números *triangulares* son definidos de la siguiente forma:



Wikipedia: [Número Triangular \(https://es.wikipedia.org/wiki/N%C3%BAmero_triangular\)](https://es.wikipedia.org/wiki/N%C3%BAmero_triangular)

- Encontrar la función de recurrencia para $T(n)$ y resolverla. (Fundamentar la construcción de la función)